

1 Task Development

On this occasion, we will use the NVIDIA CUDA API to perform pixel transformations on an array representing a 1024-pixel image with values from 0 to 255, called parallel image negation using CUDA. The idea here is to demonstrate that every pixel is independent of each other and based on that, by using parallel programming, make a thread out of each different pixel. Later, on the development of the code, we will show if the final result is correct by implementing the following equation found on the document:

$$out[i] = 255 - img[i]$$

All of this development will be supported by the package *PyCUDA* that will connect NVIDIA CUDA API to Python, done on Google Colab:

```
import pycuda.autoinit # initiates GPU
import pycuda.driver as cuda # manages memory and GPU exe
import numpy as np # creates array (img)
from pycuda.compiler import SourceModule # CUDA compile on Py
```

Important to note that it was needed to activate *GPUR4* in Colab to install dependencies and the CUDA package.

1.1 Image representation

First, let us declare our variable *img* that will contain an array of 1024 pixels (*n*) with values from 0 to 255 pixels:

```
#first, image generation as an array to focus on the CUDA application

img = np.random.randint(0, 256, size=1024).astype(np.uint8)
n = img.size
```

1.2 Memory on GPU

Then, we will declare the space for the original image and final result by using CUDA, to work better with the data we already have:

```
#memory on GPU

img_gpu = cuda.mem_alloc(img.nbytes)
out_gpu = cuda.mem_alloc(img.nbytes)
```

1.3 Data copy for GPU

```
#copy data

cuda.memcpy_htod(img_gpu, img)
```

Change of our computer as a host (CPU) to the device (GPU), sending the image to it.

1.4 CUDA Kernel

Include the function *global* indicating that this will run in the GPU. A global index called *i* will calculate pixel assignment to each thread using position, number, and size. As a result, each thread obtains one unique index *i*. Then, the condition *if*($i < n$) avoids extra threads outside the array. The main equation that applies the negative to the pixel, where each thread does it independently:

```
#CUDA kernel

kernel_code = """
__global__ void negativo(unsigned char *img, unsigned char *out, int n)
{
    int i = threadIdx.x + blockIdx.x * blockDim.x;

    if (i < n)
    {
        out[i] = 255 - img[i];
    }
}
"""
```

1.5 Kernel compilation

Converts the CUDA code into an executable, obtaining the *negativo* function ready to use:

```
mod = SourceModule(kernel_code)
negativo = mod.get_function("negativo")
```

1.6 Execution config

Block presents the group of threads, and Grid presents the group of blocks.

```
block_size = 256
grid_size = (n + block_size - 1) // block_size
```

1.7 GPU exe

Showing parallelism, the kernel is used and creates thousands of threads, where each one processes one pixel.

```
negativo(img_gpu, out_gpu, np.int32(n),
         block=(block_size,1,1),
         grid=(grid_size,1))
```

1.8 Results (dtoh)

Now, we will bring back the results by copying them from the GPU into the CPU.

dtoh = Device $\rightarrow$ Host

2 Verification

Now, with the complete implementation, it will be possible to print our results with the Original Image and Negative Image. These are the results:

```
#result verification
print("Original:", img[:10])
print("Negativo:", out[:10])

Original: [211  50 195 248 141 165  24   7  79 212]
Negativo: [ 44 205  60   7 114  90 231 248 176  43]
```

Figure 1: Comparison between Original and Negative results.

Also, for double confirmation, it is useful to implement the equation provided in the document:

```
np.all(out == 255 - img)

#to confirm the result was adequate,
#using the equation provided on the documentation

np.True_
```

Figure 2: Use of equation using *out* and *img* results

As you can see, the objective of this implementation was completed; it was possible to "turn over" each pixel value of the image by using CUDA, where each pixel performs as an independent element.